Helm Charts — What, Why, and How

Aim: Understand Helm charts end-to-end: what they are, how to create/use them, distribution, upgrades, and best practices.

You'll learn:

- What is a **chart** (and a **release**)?
- Chart structure & anatomy
- App vs **library** charts
- Values, templates, helpers
- Dependencies & subcharts
- Packaging, repositories, OCI
- Versioning, signing & provenance
- CRDs, hooks, tests
- Day-2 ops: upgrades, rollbacks

What is a Helm Chart?

A **Helm chart** is a versioned, distributable **package of Kubernetes resources**.

- Bundles: templates (YAML w/ Go templating), default values, metadata
- Installed into a cluster as a **Release** (named instance)
- Enables: reuse, parameterization, version control, lifecycle management

Chart Layout (Scaffold)

Create a starter:

```
helm create myapp
```

```
myapp/
  Chart.yaml          # metadata
  values.yaml         # default configuration
  charts/             # subchart deps
  templates/          # k8s manifests (templated)
    _helpers.tpl      # named templates/partials
    deployment.yaml
    service.yaml
    ingress.yaml
    tests/test-connection.yaml
```

> Everything in `templates/` renders to Kubernetes YAML documents.

Chart.yaml Anatomy

```yaml
apiVersion: v2           # chart API (v2 = Helm 3)
name: myapp
version: 0.2.0           # chart version (SemVer)
appVersion: "1.3.5"      # application version (display-only)
description: Demo web app chart
kubeVersion: ">=1.26"    # optional supported range
type: application         # or library
keywords: [web, demo]
sources: [https://github.com/your/repo]
icon: https://...
```

Key points

- `version` controls chart packaging & dependency ranges
- `appVersion` is for humans (e.g., container image tag)
- `type: library` creates reusable templates with no resources by itself

Application vs Library Charts

Feature	Application	Library
Renders resources	**Yes**	No (helpers/partials only)
Installable alone	**Yes**	No
Use case	Deploy apps	Share building blocks (labels, workloads)

Library example: centralize labels, common pod specs, standard annotations; consumed via `dependencies`.

Values & Overrides

Values are the **inputs** for your templates.

Sources of values (highest → lowest precedence):

1. `--set`, `--set-string`, `--set-file`
2. `-f custom.yaml` (merge left→right)
3. Chart `values.yaml`

```
helm install api ./myapp -n demo \
  --set image.tag=1.3.5,replicaCount=3 \
  -f values-demo.yaml -f values-secrets.yaml
```

> Provide production-like examples in `values.yaml` comments or a `values.example.yaml`.

Templating Basics

Use **Go templates**, **Sprig** functions, and Helm helpers.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 1 }}
  selector:
    matchLabels:
      {{- include "myapp.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "myapp.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: app
          image: {{ printf "%s:%s" .Values.image.repository (.Values.image.tag | default .Chart.AppVersion) }}
          ports:
            - containerPort: {{ .Values.service.port | default 80 }}
```

Tips: `toYaml | nindent`, helper templates in `_helpers.tpl`, `required` to fail fast.

`_helpers.tpl` (Keep Things DRY)

```
{{- define "myapp.name" -}}{{ .Chart.Name }}{{- end -}}
{{- define "myapp.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end -}}
{{- define "myapp.labels" -}}
app.kubernetes.io/name: {{ include "myapp.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
{{- end -}}
```

Use everywhere to ensure label/selector parity.

Parameterizing Common Patterns

Map → env vars

```
env:
{{- range $k, $v := .Values.env }}
- name: {{ $k }}
  value: {{ $v | quote }}
{{- end }}
```

Optional blocks

```
{{- with .Values.resources }}
resources:
  {{- toYaml . | nindent 2 }}
{{- end }}
```

Conditionally emit resources

```
{{- if .Values.ingress.enabled -}}
# ... render Ingress ...
{{- end -}}
```

Dependencies & Subcharts

Declare dependencies in `Chart.yaml`:

```yaml
dependencies:
  - name: redis
    version: "~17.0.0"
    repository: https://charts.bitnami.com/bitnami
    condition: redis.enabled
```

Pull/update:

```
helm dependency update ./myapp
```

Configure subchart via parent values:

```
redis:
  enabled: true
  architecture: standalone
```

> Use `condition` to toggle subcharts; `import-values` to pull subchart values into parent scope.

Packaging & Repositories (Classic)

```
# Lint first
helm lint ./myapp --strict

# Package (creates myapp-0.2.0.tgz)
helm package ./myapp

# Build an index for a simple static repo
echo "{}" > docs/index.yaml
helm repo index . --url https://your.github.io/helm-charts
```

Clients:

```
helm repo add your https://your.github.io/helm-charts
helm repo update
helm install myapp your/myapp --version 0.2.0
```

OCI Registries (Modern)

```
# Login
helm registry login ghcr.io -u USER -p TOKEN

# Save & push chart
helm chart save ./myapp oci://ghcr.io/your-org/myapp:0.2.0
helm chart push oci://ghcr.io/your-org/myapp:0.2.0

# Install by pulling from OCI
helm install myapp oci://ghcr.io/your-org/myapp --version 0.2.0 -n demo
```

Advantages: **immutable**, integrates with container registries, access control, provenance.

Versioning & Compatibility

- Use **SemVer** for `Chart.yaml: version`
  - `MAJOR`: breaking changes in chart behavior/values
  - `MINOR`: new features, backward-compatible
  - `PATCH`: bug fixes
- Pin subchart versions via ranges (`~`, `^`) consciously
- Document supported Kubernetes versions

Image immutability: prefer **digests** over tags in prod

```
image:
  repository: ghcr.io/org/app
  tag: sha256:abcd...  # or set digest field
```

Validating Values with JSON Schema

Add `values.schema.json` at chart root to **validate inputs**.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "replicaCount": {"type": "integer", "minimum": 1},
    "service": {
      "type": "object",
      "properties": {
        "port": {"type": "integer", "minimum": 1, "maximum": 65535},
        "type": {"enum": ["ClusterIP", "NodePort", "LoadBalancer"]}
      },
      "required": ["port", "type"]
    }
  },
  "required": ["service"]
}
```

Run:

```
helm lint ./myapp --strict
```

Tests & Hooks

Helm tests are Kubernetes Jobs/Pods labeled with `helm.sh/hook: test`.

```
apiVersion: v1
kind: Pod
metadata:
  name: "{{ include "myapp.fullname" . }}-test"
  annotations:
    "helm.sh/hook": test
spec:
  containers:
    - name: smoke
      image: curlimages/curl
      args: ["-f", "http://{{ include "myapp.fullname" . }}:{{ .Values.service.port }}/health"]
  restartPolicy: Never
```

Run:

```
helm test myapp -n demo --logs
```

Hooks orchestrate lifecycle steps (pre/post-install/upgrade). Use weights & delete policies to order/clean up.

CRDs in Charts

Options:

- Ship CRDs in a **separate CRD chart** (preferred)
- Or bundle under `crds/` folder (installed before templates, not upgraded/removed automatically)

Best practice: install/update CRDs before app chart; document this clearly.

Security & Secrets

- Avoid plain secrets in values; use **External Secrets Operator** or CSI drivers
- If you must: use **SOPS** + `helm-secrets` plugin to encrypt values files
- Enforce security context, resource limits via schema/OPA policies

```
helm plugin install https://github.com/jkroepke/helm-secrets
helm upgrade --install myapp ./myapp -f values.yaml -f secrets://values-secrets.yaml
```

Day-2: Upgrades, Rollbacks, Diff

Upgrade with new values:

```
helm upgrade myapp ./myapp -n demo -f values-prod.yaml
```

Inspect & audit:

```
helm history myapp -n demo
helm get values myapp -n demo --all
helm get manifest myapp -n demo | less
```

Safer upgrades with **diff** plugin:

```
helm plugin install https://github.com/databus23/helm-diff
helm diff upgrade myapp ./myapp -n demo -f values-prod.yaml
```

Rollback:

```
helm rollback myapp 3 -n demo
```

Distribution: Where to Find/Publish Charts

- **Artifact Hub** for discovery (community & vendor charts)
- Vendor repos (Bitnami, Elastic, Grafana Labs, etc.)
- Your org's **OCI registry** (GHCR, ACR, ECR, GCR)

Search:

```
helm search hub postgresql
helm search repo bitnami/nginx
```

Chart Signing & Provenance

Create and verify provenance files (`.prov`) with a GPG key.

```
helm package --sign --key 'KEY-ID' --keyring ~/.gnupg/secring.gpg ./myapp
helm verify myapp-0.2.0.tgz
```

Why sign?

- Integrity: chart wasn’t altered
- Authenticity: built by you

(If your org uses Cosign/Sigstore for OCI artifacts, align chart signing with it.)

Building a Chart From Scratch (Mini-Walkthrough)

1. Scaffold & clean

```
helm create web
rm -f web/templates/*
```

2. Add `deployment.yaml`, `service.yaml`, `_helpers.tpl`
3. Define `values.yaml` with `image`, `service`, `replicaCount`
4. Add optional `ingress.yaml` with `ingress.enabled`
5. `helm lint` → `helm template` → `kubeconform`
6. Install → test → iterate

Example: Service Template

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "myapp.fullname" . }}
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type | default "ClusterIP" }}
  selector:
    {{- include "myapp.selectorLabels" . | nindent 4 }}
  ports:
    - port: {{ .Values.service.port | default 80 }}
      targetPort: {{ .Values.service.port | default 80 }}
```

Example: Ingress (Conditional)

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "myapp.fullname" . }}
spec:
  ingressClassName: {{ .Values.ingress.className | default "nginx" }}
  rules:
    - host: {{ .Values.ingress.host | default (printf "%s.local" .Release.Name) }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "myapp.fullname" . }}
                port: { number: {{ .Values.service.port | default 80 }} }
{{- end }}
```

README & Documentation

Every chart should ship with:

- **README.md**: usage, parameters table, examples
- **values.yaml** with rich comments
- Version compatibility & upgrade notes (CHANGELOG)
- Maintainers & source links

> Good docs = fewer support tickets and safer consumption by others.

Best Practices Recap

- Keep charts **focused**; avoid doing everything in one chart
- DRY via helpers; ensure label/selector consistency
- Use **JSON Schema** to validate values
- Separate env values & secrets; pin versions (avoid `latest`)
- Test: `helm lint`, render, schema validation, `helm test`
- Prefer **OCI** for distribution; consider signing

Hands-On Lab (15–20 min)

1. Scaffold a chart and strip it to minimal
2. Implement Deployment, Service, optional Ingress
3. Add helpers (`name`, `fullname`, `labels`)
4. Add a simple `values.schema.json`
5. Package and install from local folder and OCI

Commands:

```
helm create shop && cd shop
helm lint --strict
helm template shop . > out.yaml
helm package . && helm chart save . oci://ghcr.io/you/shop:0.1.0
helm chart push oci://ghcr.io/you/shop:0.1.0
helm install shop oci://ghcr.io/you/shop --version 0.1.0 -n demo
```